

Вычислительная линейная алгебра

Если метод Ньютона из главы 2 был одним маленьким уравнением, упрямо повторяющимся, то линейная алгебра — это язык, на котором говорят сразу о тысячах уравнений и миллионах чисел. Этой главой мы покажем, что многие фокусы современного искусственного интеллекта — от того, как нейросеть «узнаёт» лицо человека, до того, как поисковик ранжирует страницы Интернета, — держатся всего на нескольких разложениях матриц. Мы посмотрим четыре истории: лица, котиков, Интернет и устойчивость нейросетей.

Зачем линейной алгебре отдельная глава

Вы уже встречались с матрицами в школьном курсе — как с прямоугольными таблицами чисел. В современном ИИ матрица — это **универсальный носитель данных**.

- Чёрно-белая фотография 1024×1024 — это матрица размера 1024×1024 , где каждое число — яркость пикселя от 0 (чёрный) до 255 (белый). Цветная — три такие матрицы.
- Текст из N слов словаря — это вектор частот длины N (модель «мешка слов»), и набор из M документов — матрица $M \times N$.
- Социальная сеть из N пользователей — матрица $N \times N$, где элемент (i, j) равен 1, если i подписан на j .
- Веса одного слоя нейросети — матрица W , через которую входной сигнал умножается, чтобы получить выходной.

С такими матрицами надо уметь *что-то делать*: сжимать, сравнивать, упорядочивать, обновлять. И почти всегда оказывается, что нужная операция выражается через одно и то же фундаментальное разложение матрицы — **сингулярное**. Мы изучим его и применим к четырём задачам:

1. **Узнать лицо** по фотографии — *eigenfaces* (§0.5).

2. **Сжать пространство признаков** — *eigencats* и теорема Эккарта–Янга (§0.4, §0.6).
3. **Упорядочить весь Интернет** — *PageRank* (§0.7).
4. **Понять, когда нейросеть устойчива** — *оценки Липшицевости* (§0.8).

Прежде чем перейти к сюжетам, разберёмся с одной *очень* земной проблемой: с тем, как компьютер хранит числа.

💡 $AB \neq BA$, или почему линейная алгебра не похожа на школьную

В школе вы привыкли, что от перестановки множителей произведение не меняется: $2 \cdot 3 = 3 \cdot 2$. Для квадратных матриц это *в общем случае не так* — они не коммутируют.

Самая наглядная демонстрация — повороты трёхмерного предмета. Возьмите смартфон, поверните его сначала на 90° вокруг горизонтальной оси X (вершина <<нырнула>> вперёд), потом на 90° вокруг вертикальной оси Y (предмет повернулся вправо). Теперь повторите эксперимент, но в обратном порядке. Финальные положения смартфона будут *разными*. Каждый поворот задаётся ортогональной матрицей 3×3 , и эти матрицы не коммутируют:

$$R_y(90^\circ) R_x(90^\circ) \neq R_x(90^\circ) R_y(90^\circ). \quad (1)$$

Геймеры одной из лучших игр 2023 года *The Legend of Zelda: Tears of the Kingdom* ощущают этот эффект каждый раз, когда крутят объект в инвентаре. Подробный разбор сюжета: t.me/fminxyz/15.



image

Когда арифметика лжёт: числа с плавающей точкой

Откройте Python и наберите:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Это не баг Python. То же самое выдаст любой современный язык — C, C++, Java, JavaScript. И этому есть фундаментальная причина: **двоичная система счисления** не умеет точно представить десятичную дробь 0, 1.

Двоичные дроби и почему 0, 1 — это «0, 333...»

В десятичной системе $1/3 = 0,3333...$ — бесконечная дробь. То же самое в двоичной системе случается с 0, 1:

$$0,1_{10} = 0,0001\,1001\,1001\,1001..._2 \quad (2)$$

Дробь периодическая. Любой реальный компьютер хранит её, оборвав после конечного числа разрядов (обычно 52 двоичных знака после запятой — стандарт *IEEE 754, double precision*).

Поэтому число, которое программист записывает как 0.1, на самом деле равно

$$0,1 \approx 0,100\,000\,000\,000\,000\,005\,551\,115\dots \quad (3)$$

Когда мы складываем такое «искажённое 0,1» с искажённым 0,2, ошибки усиливаются — и получается тот самый «лишний хвостик» $4 \cdot 10^{-17}$, который мы видели в листинге.

Машинная точность $\varepsilon_{\text{маш}}$

Машинной точностью называют наименьшее положительное число ε , такое что в компьютерной арифметике $1 + \varepsilon \neq 1$. Для стандарта double оно равно $\varepsilon_{\text{маш}} = 2^{-52} \approx 2,22 \cdot 10^{-16}$.

Это означает: какое бы хорошее число x вы ни записали, оно известно компьютеру лишь с относительной точностью $\sim \varepsilon_{\text{маш}}$.

Когда это становится опасным: катастрофическое сокращение

Маленькая ошибка в 17-м знаке — мелочь? Не всегда. Рассмотрим безобидное на вид вычисление:

$$g(x) = \frac{1 - \cos x}{x^2} \quad \text{при } x = 10^{-8}. \quad (4)$$

По формуле Тейлора $1 - \cos x \approx x^2/2$, так что $g(10^{-8}) \approx 0,5$. Что выдаст компьютер?

```
>>> from math import cos
>>> x = 1e-8
>>> (1 - cos(x)) / x**2
0.0      # вместо 0.5!
```

Что произошло? Число $\cos(10^{-8}) \approx 1 - 5 \cdot 10^{-17}$ так близко к единице, что компьютер просто хранит его как *ровно единицу* (поправка $5 \cdot 10^{-17}$ меньше $\varepsilon_{\text{маш}}$). Разность $1 - \cos x$ обнулилась, и весь ответ обнулится вместе с ней.

Это явление называется **катастрофическим сокращением** (*catastrophic cancellation*): когда из двух близких чисел вычитают и теряют все значащие цифры разом.

Пример 0.1. Спасение через переписывание формулы

Воспользуемся тождеством $1 - \cos x = 2 \sin^2(x/2)$:

$$g(x) = \frac{2 \sin^2(x/2)}{x^2} = \left(\frac{\sin(x/2)}{x/2} \right)^2. \quad (5)$$

Внутри теперь нет вычитания близких чисел, и Python выдаёт 0.4999999999999983 — погрешность лишь в последнем знаке.

Важно

Главный практический вывод. Если ваш численный алгоритм выдаёт странный ответ, первая гипотеза, которую следует проверить, — это не «баг в библиотеке», а *катастрофическое сокращение* в вашей формуле. Часто его можно убрать, переписав формулу алгебраически эквивалентным, но численно устойчивым способом.

Историческая справка

Стандарт *IEEE 754* был принят в 1985 г., в значительной мере благодаря Уильяму Кэхэну (*William Kahan*, премия Тьюринга 1989 г.). До этого каждый производитель процессоров делал плавающую арифметику по-своему, и одна и та же программа на разных машинах могла давать заметно разные ответы. Кэхэн также является автором знаменитого **алгоритма компенсированного суммирования**, который позволяет складывать миллионы чисел с почти двойной точностью.

Сингулярное разложение *SVD*: главный фокус линейной алгебры

Геометрическая идея

Возьмём любую матрицу A размера $m \times n$ и применим её к единичному кругу в $\mathbb{R}^n$ (то есть к множеству всех векторов x с $\|x\| = 1$, где $\|\cdot\|$ — обычная евклидова длина). Что получится в $\mathbb{R}^m$?

Теорема 0.2. Сингулярное разложение (SVD)

Для любой вещественной матрицы A размера $m \times n$ существует разложение

$$A = U \Sigma \bar{V} \quad (6)$$

где

- U — ортогональная матрица $m \times m$ (поворот в $\mathbb{R}^m$);
- V — ортогональная матрица $n \times n$ (поворот в $\mathbb{R}^n$);
- Σ — «диагональная» матрица $m \times n$ с неотрицательными числами $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ на диагонали (и нулями ниже), где $r = \text{rank } A$.

Числа σ_i называются **сингулярными числами** матрицы A .

Содержательно (0.1) говорит: *любая линейная операция* — это три простых действия подряд:

$$\underbrace{\bar{V}}_{\text{поворот в исходном пространстве}} \rightarrow \underbrace{\Sigma}_{\text{растяжение по координатным осям}} \rightarrow \underbrace{U}_{\text{поворот в новом пространстве}} \quad (7)$$

Геометрически это значит: образ единичной сферы под действием A — это **эллипсоид**, у которого полуоси равны $\sigma_1, \sigma_2, \dots$ (см. рис. 0.1).

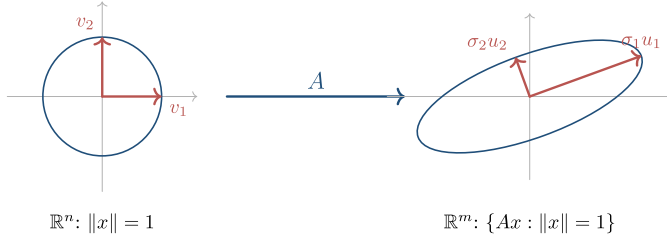


Рис. 0.1. SVD геометрически. Матрица A переводит единичный круг в эллипс, полуоси которого равны сингулярным числам σ_1, σ_2 , а направления полуосей — столбцы матрицы U (левые сингулярные векторы). Векторы v_1, v_2 (столбцы V , правые сингулярные векторы) — это те направления в исходном пространстве, которые A переводит в полуоси эллипса.

Связь с собственными значениями

Сингулярные числа — это, по сути, переодетые собственные значения. A именно:

$$\bar{A}A = (U\Sigma\bar{V})(U\Sigma\bar{V}) = V\bar{\Sigma}\Sigma\bar{V}. \quad (8)$$

Матрица $\bar{\Sigma}\Sigma$ — диагональная, на её диагонали стоят $\sigma_1^2, \sigma_2^2, \dots$. Значит, σ_i^2 — это *собственные значения* симметричной матрицы $\bar{A}A$, а столбцы V — её *собственные векторы*. Эта формула важна нам по двум причинам: (1) она обосновывает существование SVD (через известную спектральную теорему для симметричных матриц), (2) она будет рабочим инструментом в § 0.5, когда мы будем строить базис «собственных лиц» (eigenfaces).

💡 Историческая справка

Сингулярное разложение независимо открыли итальянский математик *Эудженио Бельтрами* (1873 г.) и француз *Камилл Жордан* (1874 г.). В современном виде, как разложение прямоугольной матрицы, его сформулировал *Карл Эккарт* с *Гейлом Янгом* в 1936 г. Численно надёжный алгоритм вычисления SVD появился лишь в 1965 г. (Голуб и Кахан) — и стал одним из ключевых алгоритмов XX века. Сегодня он реализован в любой математической библиотеке: `numpy.linalg.svd`, `scipy.linalg.svd`, `torch.svd`.

Лучшее малоранговое приближение: теорема Эккарта–Янга

Самое замечательное свойство SVD состоит в том, что оно позволяет *ужимать* матрицу, выбрасывая всё «несущественное».

Разложение по слагаемым ранга один

Перепишем (0.1) по-другому. Обозначим столбцы матрицы U через $u_1, \dots, u_m$ (это *левые сингулярные векторы*), а столбцы V — через $v_1, \dots, v_n$ (*правые сингулярные векторы*). Тогда A есть сумма r слагаемых, каждое из которых имеет ранг один:

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T. \quad (9)$$

Каждое слагаемое $u_i v_i^T$ — это «вертикальный вектор умножить на горизонтальный», и оно само по себе является матрицей ранга один (рис. 0.2).

$$\begin{array}{c} u \\ m \times 1 \end{array} \times \begin{array}{c} v^T \\ 1 \times n \end{array} = \begin{array}{c} uv^T \\ m \times n, \text{ ранг } 1 \end{array}$$

Рис. 0.2. Матрица ранга один: каждый столбец результата пропорционален u , каждая строка — пропорциональна v . Чтобы её хранить, достаточно $m + n$ чисел вместо mn .

Если оборвать сумму (0.3) на k -м слагаемом ($k < r$), получим приближение:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T. \quad (10)$$

Это матрица того же размера $m \times n$, но ранга всего лишь k .

Теорема Эккарта–Янга: A_k — наилучшее приближение

⚠ Теорема 0.3. Эккерт–Янг–Мирский (1936/1960)

Среди всех матриц B размера $m \times n$ ранга не более k матрица A_k из (0.4) даёт *наименьшую* ошибку в смысле спектральной нормы:

$$\min_{\text{rank } B \leq k} \|A - B\|_2 = \|A - A_k\|_2 = \sigma_{k+1}. \quad (11)$$

Здесь $\|M\|_2 = \max_{\|x\|=1} \|Mx\| = \sigma_1(M)$ — **спектральная норма** матрицы M (равна её наибольшему сингулярному числу). Утверждение остаётся в силе и для *фробениусовой* нормы $\|M\|_F = \sqrt{\sum_{i,j} m_{ij}^2}$:

$$\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \sigma_{k+2}^2 + \dots + \sigma_r^2}. \quad (12)$$

Смысл теоремы: *если хочется сжать матрицу до ранга k , лучшее, что вы можете сделать, — оставить первые k компонент SVD*. Ошибка контролируется *отброшенным* сингулярным числом σ_{k+1} .

Пример: сжатие изображения

Возьмём фотографию 1024×768 в градациях серого. Хранение «в лоб» — это $1024 \cdot 768 = 786\,432$ чисел. Применим к матрице фото SVD и оставим только k старших компонент: придётся хранить

$$k \cdot (1024 + 768) + k = k \cdot 1793 \quad (13)$$

чисел — по одному столбцу u_i , одной строке v_i^T и одному числу σ_i на каждое слагаемое. Уже при $k = 50$ это $\sim 90\,000$ чисел — в *восемь* раз меньше исходного, а визуально потеря почти незаметна (рис. 0.3).

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, color

# Load and convert to grayscale
img = color.rgb2gray(data.astronaut()) # shape (512, 512)
U, S, Vt = np.linalg.svd(img, full_matrices=False)
```

```
# Reconstruct using only top-k singular components
ranks = [1, 5, 20, 50, 200]
fig, axes = plt.subplots(1, len(ranks)+1, figsize=(14, 3))
axes[0].imshow(img, cmap='gray'); axes[0].set_title('original')
for ax, k in zip(axes[1:], ranks):
    A_k = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
    ax.imshow(A_k, cmap='gray')
    ax.set_title(f'rank {k}')
for ax in axes: ax.axis('off')
plt.tight_layout(); plt.show()
```

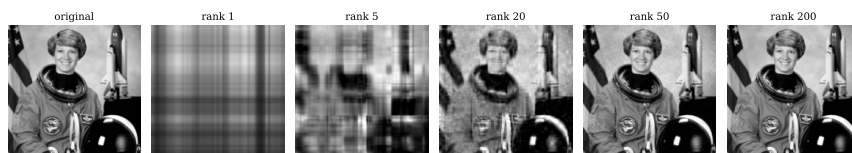


Рис. 0.3. Малоранговое приближение портрета астронавта. Уже при ранге 50 изображение визуально неотличимо от оригинала; при ранге 200 восстановление идеально. По теореме Эккарта–Янга это *лучшее* приближение фиксированного ранга.

А если картинка цветная? Цветное изображение $H \times W \times 3$ можно сжать тремя способами: (1) поканально, считая SVD отдельно для R, G, B; (2) перевести в $YCbCr$ и сжимать только канал яркости Y ; (3) «развернуть» изображение в матрицу $H \times 3W$ и применить одну SVD. В простейшем поканальном варианте для ранга r нужно хранить $3r(H + W + 1)$ чисел вместо $3HW$:



Рис. 0.4. Цветную фотографию сжимаем поканально: SVD ранга r для каждого из R, G, B и склейка. Под каждым кадром — доля памяти от оригинала. Варианты для цвета: t.me/fminxyz/16.

i LoRA: малоранговая адаптация больших моделей

В 2021 г. Эдвард Ху с соавторами заметили: когда большую языковую модель дообучают на новую задачу, *обновление* матриц весов ΔW оказывается почти малоранговой матрицей. Поэтому вместо обучения «полной» ΔW можно сразу искать её в виде $\Delta W = U\bar{V}$ с маленьким k , где U, V — тонкие прямоугольники.

Это метод **LoRA** (*Low-Rank Adaptation*). Для модели с 175 миллиардами параметров такой приём сокращает объём дообучаемых весов в тысячи раз — и именно благодаря ему сегодня можно «дофайнтюнить» большую модель на бытовой видеокарте.

Теоретический фундамент LoRA — та самая теорема Эккарта–Янга 0.3, которой почти 90 лет.

Eigenfaces: как алгоритм узнаёт лицо

Постановка задачи

В 1991 г. Мэттью Тёрк и Алекс Пентланд из MIT поставили простой вопрос: *можно ли обучить компьютер узнавать лицо, не прибегая ни к каким специальным «лицевым» признакам — носу, глазам, контурам, — а просто работая с матрицей пикселей?* Их ответ — метод **собственных лиц** (*eigenfaces*) — стал классикой и открыл целую область computer vision.

Идея: каждая фотография лица — это точка в очень многомерном пространстве. Например, лицо 64×64 — это вектор длины $64 \cdot 64 = 4096$ в пространстве $\mathbb{R}^{4096}$. Двух разных людей снимали много раз — получаем облако точек в этом пространстве.

Главное наблюдение: *это облако очень узкое*. Хотя пространство 4096-мерное, реальные лица занимают в нём подпространство размерности всего лишь ~ 50 . Найти это подпространство — и есть задача.

Olivetti Faces: 40 человек $\times$ 10 снимков (показано по 1)



Рис. 0.5. Фрагмент датасета *Olivetti Faces*: 400 фотографий 64×64 пикселя, 40 человек по 10 снимков каждого. Каждая фотография — точка в $\mathbb{R}^{4096}$.

Шаг 1. Среднее лицо и центрирование

Пусть у нас N фотографий $x_1, \dots, x_N \in \mathbb{R}^d$ (где $d = 4096$ для 64×64). Вычислим **среднее лицо**:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i. \quad (14)$$

И вычтем его из каждого изображения: $\tilde{x}_i = x_i - \bar{x}$. *Среднее лицо* (рис. 0.6) — это размытая маска, в которой видно «лицо вообще»: усреднённые глаза, нос, рот, овал. Дальше мы будем работать только с отклонениями от этого среднего.



Рис. 0.6. Среднее лицо по 400 снимкам датасета Olivetti.

Шаг 2. Главные направления изменчивости

Соберём центрированные изображения в строки матрицы $X \in \mathbb{R}^{N \times d}$ (одна строка = одно лицо). Применим к ней SVD:

$$X = U \Sigma \bar{V}. \quad (15)$$

Каждая *строка* $\bar{v}_i$ матрицы $\bar{V}$ — это вектор длины $d = 4096$, который можно нарисовать как картинку 64×64 . Эти картинки и называются **собственными лицами** — eigenfaces (рис. 0.7). Содержательно:

- v_1 — направление наибольшей изменчивости. Чаще всего это «освещение слева/справа».
- v_2 — направление второй по величине изменчивости (часто «улыбка / не улыбка»).
- $v_3, \dots$ — постепенно более тонкие признаки: очки, форма щёк, наклон головы, индивидуальные особенности.

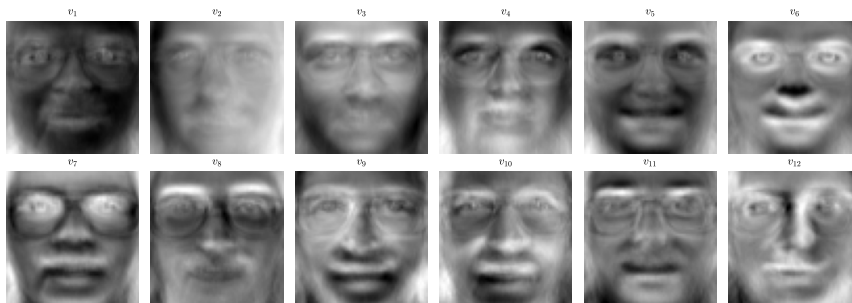


Рис. 0.7. Первые 12 eigenfaces датасета Olivetti, отрисованные как картинки 64×64 . Эти странные «полу-лица» — координатные оси в пространстве, где живут все настоящие лица.

Шаг 3. Лицо как вектор из 50 чисел

Зафиксируем k (обычно $k = 30 \div 100$) и спроектируем каждое лицо x на эти k собственных лиц:

$$\alpha_i = \bar{v}_i^\top (x - \bar{x}), \quad i = 1, \dots, k. \quad (16)$$

Получили вектор коэффициентов $\alpha = (\alpha_1, \dots, \alpha_k) \in \mathbb{R}^k$. Это и есть «лицо человека» с точки зрения алгоритма — всего 50 чисел вместо 4096.

Шаг 4. Распознавание и реконструкция

Распознавание. Чтобы узнать, кто на новой фотографии y , вычислим её коэффициенты β по (0.6) и найдём в базе ближайшего соседа: $\hat{j} = \arg \min_j \| \beta - \alpha^{(j)} \|$.

Реконструкция. Зная α , можно восстановить лицо:

$$x \approx \bar{x} + \sum_{i=1}^k \alpha_i v_i. \quad (17)$$

Рис. 0.8 показывает, как качество реконструкции растёт с k .

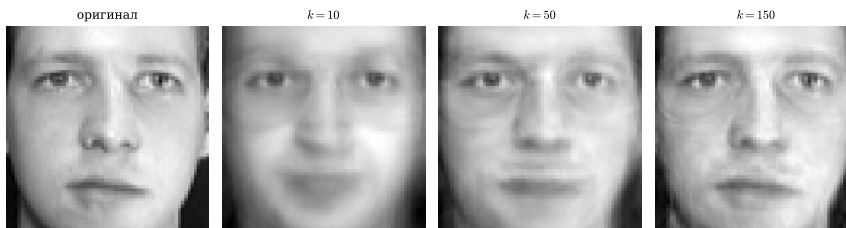


Рис. 0.8. Реконструкция одной из фотографий датасета через первые k eigenfaces. При $k = 10$ узнаётся «общий тип» лица; при $k = 50$ — уже конкретный человек; при $k = 150$ — почти оригинал.

Реализация на Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces

# 1. Load: 400 faces, 64x64 each
faces = fetch_olivetti_faces().images # (400, 64, 64)
X = faces.reshape(400, -1) # (400, 4096)

# 2. Center: subtract the mean face
x_bar = X.mean(axis=0)
X_centered = X - x_bar

# 3. SVD => eigenfaces are rows of Vt
U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)

# 4. Project all faces onto top-k eigenfaces
k = 50
alpha = X_centered @ Vt[:k].T # (400, 50)

# 5. Recognise: nearest neighbour in alpha-space
def recognise(y, k=50):
    y_centered = y.flatten() - x_bar
    beta = Vt[:k] @ y_centered
    distances = np.linalg.norm(alpha - beta, axis=1)
    return distances.argmin()

# 6. Reconstruct a face from k coefficients
def reconstruct(idx, k=50):
    return x_bar + Vt[:k].T @ alpha[idx, k]
```

💡 Это интересно

Метод eigenfaces — родоначальник всех современных систем распознавания лиц, включая ту, что разблокирует ваш смартфон. Современные системы заменяют SVD на свёрточную нейросеть (см. § 3.3.7), но базовая идея — *сжать лицо в короткий вектор, в котором близкие лица оказываются рядом* — осталась той же. Эти короткие векторы сегодня называют **эмбедингами**.

Eigencats: тот же приём для котов*

Метод eigenfaces работает на любых однородных объектах, не только на лицах. Возьмём *Cats Faces Dataset*: 29 843 фотографии котов 64×64 пикселя. Векторизуем — и получим матрицу $A \in \mathbb{R}^{29\,843 \times 4096}$. Сингулярные векторы матрицы $\bar{A}\bar{A}$ называют **eigencats**.



Рис. 0.9. Малоранговое восстановление четырёх котов на разных рангах r (анимация: t.me/fminxyz/12).

Как и в случае с лицами (§0.5), сами строки матрицы $\bar{V}$ образуют *базис* пространства котов. Каждая такая строка — собственный вектор размерности 4096, который можно нарисовать как картинку 64×64 . Эти картинки и называют **eigencats**.

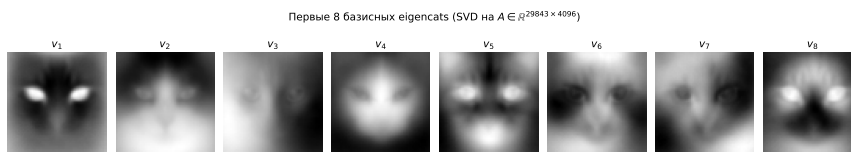


Рис. 0.10. Первые восемь eigencats — строки $\bar{V}$, свёрнутые в квадраты 64×64 . v_1 ловит освещение и общую форму морды, v_2 — контраст верх/низ (наклон), v_3 — лево/право, дальше — всё более тонкие особенности.

В сжатом виде каждый кот превращается в ~ 50 чисел: коэффициенты $\alpha_i = \bar{v}_i(x - \bar{x})$. Этого хватает, чтобы потом восстановить узнаваемое изображение и отличать одного кота от другого.

i Что это нам говорит о ИИ

Пространство «реальных» объектов всегда узкое. Будь это лица людей, морды котов, рукописные цифры (датасет MNIST) или фотографии природы — реальные данные занимают в исходном многомерном пространстве *маленькое* подпространство. Распознавание = найти это подпространство и работать в нём.

Эта идея — фундамент современных автокодировщиков и эмбедингов (см. § 3.4.3–3.4.6). SVD — её самая ранняя и самая понятная реализация.

PageRank: как Google нашёл порядок в Интернете

Задача и история

К 1996 г. поисковики типа *AltaVista* и *Yahoo!* ранжировали страницы по тому, сколько раз искомое слово встречается в тексте. Это легко обманывалось: достаточно было набить страницу популярными словами белым цветом по белому фону, и она попадала в топ выдачи.

Лоуренс Пейдж и Сергей Брин, тогда аспиранты Стэнфорда, предложили другой принцип ранжирования: *страница важна, если на неё ссылаются другие важные страницы.*

Граф ссылок как матрица

Обозначим N — число всех страниц Интернета (на 1998 г. — около $2,4 \cdot 10^8$, сегодня — $\sim 10^{11}$). Построим матрицу $P \in \mathbb{R}^{N \times N}$:

$$P_{ij} = \begin{cases} \frac{1}{c_j}, & \text{если со страницы } j \text{ есть ссылка на } i, \\ 0, & \text{иначе,} \end{cases} \quad (18)$$

где c_j — полное число исходящих ссылок со страницы j . Содержательно: P_{ij} — это вероятность того, что пользователь, сидящий на странице j и случайно ткнувший на одну из её ссылок, окажется на странице i . Поэтому P называется матрицей переходов **случайного блуждания** по графу Интернета.

Стационарное распределение = собственный вектор

Пусть $r \in \mathbb{R}^N$ — вектор «важностей»: r_i — степень важности страницы i . Принцип Пейджа–Брина формализуется системой:

$$r = Pr. \quad (19)$$

То есть важность каждой страницы равна сумме важностей тех страниц, которые на неё ссылаются, делённой на число их исходящих ссылок.

Уравнение (0.7) говорит: r — это *собственный вектор* матрицы P с собственным значением 1. И в этом — вся суть PageRank.

⚠ Теорема 0.4. Перрон–Фробениус для PageRank

Если все элементы матрицы P положительны (или хотя бы граф ссылок сильно связан и неперiodичен), то собственное значение 1 у P существует, оно *простое* и наибольшее по модулю, а соответствующий собственный вектор r единственен и имеет положительные компоненты. Этот r называется **стационарным распределением** случайного блуждания.

(Эта теорема — Оскар Перрон, 1907 г., и Георг Фробениус, 1912 г. — исторически появилась задолго до Интернета, в контексте моделей популяций.)

Демпфирующий множитель и формула Брина–Пейджа

В реальной матрице Интернета у некоторых страниц нет исходящих ссылок (*dangling pages*), а в других группах — циклы. Чтобы гарантировать применимость теоремы Перрона–Фробениуса, Пейдж и Брин ввели **демпфирующий множитель** $\beta \in (0, 1)$ (обычно $\beta = 0,85$):

$$r = \beta P r + \frac{1 - \beta}{N} \mathbf{1}. \quad (20)$$

Здесь $\mathbf{1} = (1, 1, \dots, \bar{1})$. Содержательно: с вероятностью $\beta \approx 0,85$ случайный сёрфер кликает по ссылке, а с вероятностью $1 - \beta \approx 0,15$ — телепортируется на случайную страницу Интернета.

Как находить r : степенной метод

Прямо решать систему $(I - \beta P)r = \frac{1 - \beta}{N} \mathbf{1}$ с $N \sim 10^{11}$ невозможно. Но и не надо: вектор r можно получить простой итерацией.

! Алгоритм

Степенной метод для PageRank.

1. Инициализация: $r^{(0)} \leftarrow \frac{1}{N} \mathbf{1}$.
2. Итерация: $r^{(t+1)} \leftarrow \beta P r^{(t)} + \frac{1 - \beta}{N} \mathbf{1}$.
3. Остановка: $\| r^{(t+1)} - r^{(t)} \| < \varepsilon$.

Сходимость гарантируется теоремой Перрона–Фробениуса: $\|r^{(t)} - r\| \leq C \cdot \beta^t$, то есть погрешность убывает геометрически со знаменателем $\beta = 0,85$. На практике хватает $50 \div 100$ итераций.

Игрушечный пример: 6 страниц

Возьмём маленький «Интернет» из шести страниц (рис. 0.11) и посчитаем для него PageRank.

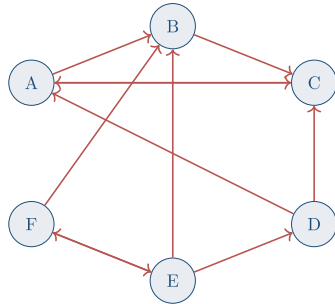


Рис. 0.11. Игрушечный «Интернет» из 6 страниц со ссылками. Видно, например, что на страницу С ссылаются три другие, а на F — только одна; будет ли С лидером по PageRank?

```

import numpy as np

# Adjacency list -> column-stochastic transition matrix P
links = {'A': ['B', 'C', 'D', 'E'], 'B': ['A', 'C'], 'C': ['A', 'B', 'D'], 'D': ['C', 'E', 'F'], 'E': ['A', 'D', 'F'], 'F': ['D', 'E']}
pages = list(links); n = len(pages); idx = {p: i for i, p in enumerate(pages)}

P = np.zeros((n, n))
for j, p in enumerate(pages):
    out = links[p]
    for q in out:
        P[idx[q], j] = 1.0 / len(out)

# PageRank via power iteration
beta, eps = 0.85, 1e-10
r = np.ones(n) / n
for t in range(200):
    r_new = beta * P @ r + (1 - beta) / n
    if np.linalg.norm(r_new - r, 1) < eps: break
    r = r_new

for p, ri in sorted(zip(pages, r), key=lambda t: -t[1]):
    print(f'{p}: {ri:.4f}')
    
```

Выдача программы:

C: 0,295, A: 0,252, B: 0,175, E: 0,098, F: 0,092, D: 0,087.

Лидер — C, хотя на неё формально ссылаются столько же страниц, сколько на A: PageRank учёл, что одна из ссылок на C приходит с уже-важной страницы B.

💡 Историческая справка

Алгоритм PageRank был опубликован в 1998 г. в студенческой работе *Sergey Brin, Larry Page*, «*The PageRank Citation*»

Ranking: Bringing Order to the Web». В том же году Брин и Пейдж основали Google. К 2004 г. компания вышла на IPO, а к 2010-м стала одной из крупнейших в мире. Сам Стэнфорд получил ~ 336 миллионов долларов от лицензирования патента — который изначально просто описывал итерационный метод для собственного вектора.

Идея использовать собственный вектор графа для измерения «важности» позднее была обобщена и применена в наукометрии (*Eigenfactor* для ранжирования научных журналов), в рекомендательных системах, а также в современных нейросетях для графов (*Graph Neural Networks*).

Когда нейросеть устойчива: оценки Липшицевости*

Сюжет: когда панда становится гиббоном

В 2014 г. Иэн Гудфеллоу и соавторы опубликовали ставшую знаменитой картинку (рис. 0.12): обычная фотография панды, которую современная нейросеть уверенно опознаёт как «panda» с вероятностью 57,7%. К каждому пикселю прибавлена крошечная, визуально незаметная человеку поправка. На обновлённой картинке сеть теперь уверенно (с вероятностью 99,3%) считает, что видит *гиббона*.

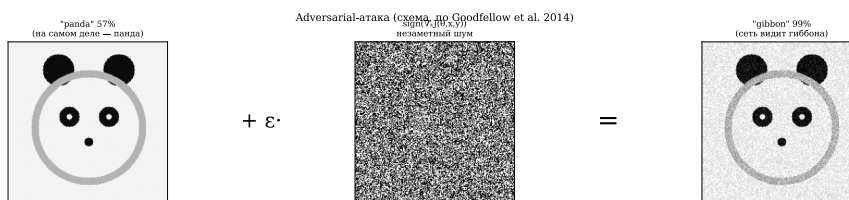


Рис. 0.12. Adversarial-атака на нейросеть (Goodfellow et al., 2014). Слева — оригинал, классифицирован как «panda» с уверенностью 57,7%. В центре — почти невидимое возмущение, масштабированное для наглядности. Справа — атакованный пример, классифицируется как «gibbon» с уверенностью 99,3%.

Этот эффект называется **adversarial-атакой** и долгое время считался экзотикой. Сейчас понятно: причина в простом числе, которое сопровождает любую функцию — её *константе Липшица*.

Константа Липшица: формальное определение

i Определение 0.5. Константа Липшица функции

Функция $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ называется **Липшицевой** с константой $L > 0$, если для любых $x, y \in \mathbb{R}^n$ выполнено

$$\|f(x) - f(y)\| \leq L \|x - y\|. \quad (22)$$

Наименьшее L , для которого (0.9) выполнено, называется *константой Липшица* функции f и обозначается $\text{Lip}(f)$.

Содержательно $\text{Lip}(f)$ показывает, во сколько раз может усииться входное возмущение на выходе функции. Если $\text{Lip}(f) = L$, то малое возмущение δ на входе может вызвать возмущение до $L \cdot \|\delta\|$ на выходе.

Пример. Для линейного отображения $f(x) = Wx$ имеем $f(x) - f(y) = W(x - y)$, откуда $\|f(x) - f(y)\| \leq \|W\|_2 \cdot \|x - y\|$, и константа Липшица — это в точности спектральная норма матрицы W :

$$\text{Lip}(Wx) = \|W\|_2 = \sigma_1(W). \quad (23)$$

Здесь $\sigma_1(W)$ — наибольшее сингулярное число матрицы W . Вот здесь в нашу историю входит SVD из § 0.3.

Композиция: Липшицева константа цепочки

⚠ Теорема 0.6. Композиция Липшицевых функций

Если f и g — Липшицевы с константами L_f и L_g , то их композиция $f \circ g$ — тоже Липшицева, причём

$$\text{Lip}(f \circ g) \leq \text{Lip}(f) \cdot \text{Lip}(g). \quad (24)$$

Доказательство. $\|f(g(x)) - f(g(y))\| \leq L_f \|g(x) - g(y)\| \leq L_f L_g \|x - y\|$. $\square$

Применение к нейросети

Возьмём типичную нейросеть глубины K :

$$f(x) = \sigma(W_K \sigma(W_{K-1} \sigma(\dots \sigma(W_1 x))), \quad (25)$$

где W_i — матрицы весов, σ — поэлементная активация (ReLU, сигмоида, и т.п.). Для большинства активаций $\text{Lip}(\sigma) \leq 1$ (для ReLU и сигмoиды это равно 1). По теореме 0.6:

$$\text{Lip}(f) \leq \prod_{i=1}^K \|W_i\|_2 \quad (26)$$

Это и есть тот «лишний рычаг», который объясняет уязвимость нейросетей. Если $\|W_i\|_2 \approx 2$ для каждого слоя из $K = 20$ слоёв, то возмущение на входе *потенциально* усиливается в $2^{20} \approx 10^6$ раз. Невидимая поправка δ с $\|\delta\| = 10^{-3}$ может вызвать изменение выхода до 10^3 — этого хватит, чтобы перебросить классификатор из одного класса в другой.

Как защищаться: спектральная нормализация

Естественная стратегия: после каждого шага обучения нормализовать матрицу W_i так, чтобы $\|W_i\|_2 = 1$. Это и есть метод **Spectral Normalization** (Миято и соавт., 2018), сделавший обучение генеративных нейросетей (GAN) намного стабильнее.

Технически: на каждом шаге считаем $\sigma_1(W)$ (с помощью одной итерации степенного метода — почти бесплатно!) и заменяем $W \mapsto W/\sigma_1(W)$.

```
def spectral_normalize(W, n_iter=1):
    """One step of power iteration to estimate sigma_1(W) and rescale."""
    u = np.random.randn(W.shape[0])
    for _ in range(n_iter):
        v = W.T @ u; v /= np.linalg.norm(v)
        u = W @ v; u /= np.linalg.norm(u)
    sigma_1 = u @ W @ v
    return W / sigma_1
```

QR-алгоритм: как находят все собственные числа*

Зачем нужен ещё один метод

В §0.7 мы использовали *степенной метод*, чтобы найти *один* собственный вектор — доминантный, соответствующий наибольшему по модулю собственному числу. А что, если нужны *все* собственные числа сразу — например, чтобы найти главные компоненты k -ранговой аппроксимации, посчитать спектральную норму или определить устойчивость линейной системы $\dot{x} = Ax$ (она устойчива, когда все собственные числа имеют отрицательную действительную часть)?

Здесь и нужен **QR-алгоритм** — одна из жемчужин численных методов. Описан в 1961 году Джоном Фрэнсисом и независимо Верой Кубланской, по сей день остаётся *рабочей*

💡 Это интересно

Связь с устойчивостью в физике. Похожая ситуация хорошо известна в теории динамических систем: если константа Липшица отображения > 1 , малое возмущение начального условия экспоненциально нарастает (классический эффект бабочки в погодных моделях). Spectral Normalization можно рассматривать как искусственное гашение этого эффекта.

лошадкой библиотек LAPACK и NumPy — именно его вы вызываете каждый раз, когда пишете `numpy.linalg.eig`.

Сам алгоритм: две строчки

Идея до неприличия проста. Возьмём QR-разложение матрицы A : $A = QR$, где Q ортогональная (поворот), R верхнетреугольная. Теперь переставим сомножители:

$$A_{k+1} = R_k Q_k, \quad \text{где } Q_k R_k = \text{qr}(A_k). \quad (27)$$

И повторим. Всё.

! QR-алгоритм

Вход: квадратная матрица $A_0 \in \mathbb{R}^{n \times n}$, число итераций N .

Повторять $k = 0, 1, \dots, N - 1$:

1. Вычислить QR-разложение: $Q_k, R_k \leftarrow \text{qr}(A_k)$.
2. Обновить: $A_{k+1} \leftarrow R_k Q_k$.

Возврат: A_N — (почти) верхнетреугольная; на её диагонали стоят собственные числа исходной A .

Что происходит и почему это работает

Все матрицы A_k подобны исходной: $A_{k+1} = \bar{Q}_k A_k Q_k$. А подобные матрицы имеют одинаковые собственные числа (это базовый факт линейной алгебры). При этом с каждой итерацией поддиагональные элементы A_k уменьшаются в геометрической прогрессии, и в пределе A_k становится верхнетреугольной — значит, на её диагонали как раз и стоят собственные числа.

⚠ Теорема 0.7. Сходимость QR-алгоритма

Пусть собственные числа A действительны и различны по модулю: $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$. Тогда последовательность A_k , полученная QR-итерациями, сходится к верхнетреугольной матрице, на диагонали которой стоят $\lambda_1, \lambda_2, \dots, \lambda_n$ (в порядке убывания модуля), причём поддиагональные элементы убывают как $|\lambda_{i+1}/\lambda_i|^k$.

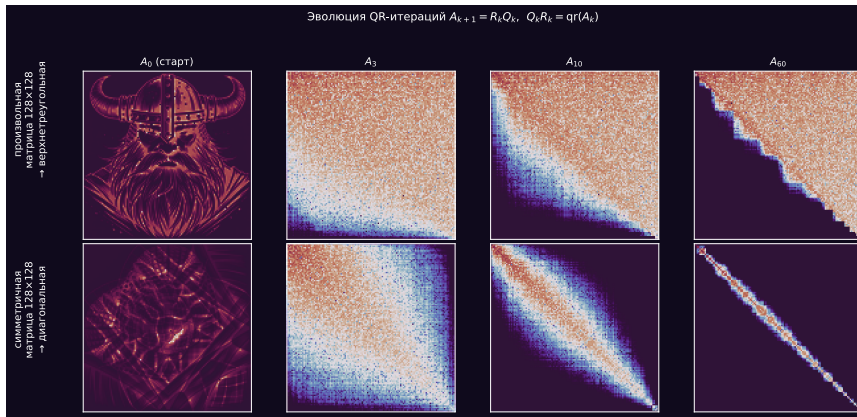


Рис. 0.13. Сходимость QR-алгоритма на 5×5 -матрицах. Верхний ряд: произвольная (несимметричная) матрица сходится к верхнетреугольной с собственными числами на диагонали (4, -3, 2.5, -1.5, 1). Нижний ряд: симметричная матрица сходится к диагональной — это полезное свойство симметрии. Источник: t.me/fminxyz/30.

Симметричный случай: сходится к диагональной

Если исходная A симметрична, $\bar{A} = A$, то все промежуточные A_k тоже симметричны, и сходимость идёт не просто к верхнетреугольной, а сразу к *диагональной* матрице. На рис. 0.13 (нижний ряд) видно, что внедиагональные элементы зануляются с обеих сторон одновременно. Это особенно ценно: получив диагональ, мы сразу имеем *и* собственные числа, *и* собственные векторы (произведение всех Q_k сходится к матрице собственных векторов).

```
import numpy as np

def qr_eigvals(A, n_iters=200):
    A = A.astype(float).copy()
    for _ in range(n_iters):
        Q, R = np.linalg.qr(A)
        A = R @ Q
    return np.diag(A)  # собственные числа

# Проверка на случайной симметричной матрице:
np.random.seed(0)
M = np.random.randn(5, 5)
S = (M + M.T) / 2
print(sorted(qr_eigvals(S), reverse=True))
print(sorted(np.linalg.eigvalsh(S).tolist(), reverse=True))
```

Реальный QR в LAPACK

В практических реализациях используют *двухступенчатый* вариант: сначала A приводят к **форме Хессенберга** (почти треугольной — только одна поддиагональ ненулевая) ортогональными преобразованиями Хаусхолдера, а потом запускают QR-итерации со сдвигами (*Wilkinson shift*). Это даёт сходимость за $\mathcal{O}(n)$ итераций вместо

$\mathcal{O}(n \log n)$ <<наивного>> QR, и общая сложность получается $\mathcal{O}(n^3)$ — тот же порядок, что и у SVD.

Связь с SVD

QR и SVD — родственники. Сингулярные числа матрицы A совпадают с квадратными корнями собственных чисел матрицы $\bar{A}A$. Поэтому один из стандартных способов вычислить SVD — сначала построить $\bar{A}A$ (она симметрична), применить QR-алгоритм и получить $\sigma_i = \sqrt{\lambda_i}$. На практике, однако, так *не делают*: построение $\bar{A}A$ возводит в квадрат число обусловленности и теряет точность. Современные алгоритмы SVD (например, *Голуба–Кахана*) применяют QR-итерации напрямую к би-диагональной форме A , обходя $\bar{A}A$.

Подведение итогов главы

- Матрица — это универсальный носитель данных для ИИ: картинки, тексты, графы, веса нейросети.
- **Числа с плавающей точкой** (§0.2) — вычисления на компьютере всегда приближённые с относительной ошибкой $\sim \varepsilon_{\text{маш}} \approx 10^{-16}$. Самая опасная ошибка — *катастрофическое сокращение* при вычитании близких чисел; её часто можно устранить, переписав формулу алгебраически эквивалентным способом.
- **Сингулярное разложение** $A = U\Sigma\bar{V}$ (§0.3) представляет любую матрицу как «поворот $\rightarrow$ растяжение по осям $\rightarrow$ поворот». Сингулярные числа σ_i — это коэффициенты растяжения.
- **Теорема Эккарта–Янга** (§0.4): $A_k = \sum_{i \leq k} \sigma_i u_i \bar{v}_i$ — лучшее приближение ранга k матрицы A . На этой теореме стоит сжатие изображений, метод *eigenfaces* и современный LoRA для дообучения больших языковых моделей.
- **Eigenfaces / eigencats** (§0.5–0.6): реальные данные (лица, коты, цифры) живут в очень узком подпространстве своего номинального многомерного пространства. Найти это подпространство = применить SVD к выборке.

- **PageRank** (§0.7): «важность» страницы Интернета = компонента собственного вектора матрицы переходов случайного блуждания. Алгоритм находит его простой итерацией; теорема Перрона–Фробениуса гарантирует сходимость.
- **Константа Липшица** нейросети (§0.8) растёт мультипликативно от слоя к слою; именно поэтому глубокие сети уязвимы к adversarial-атакам. Спектральная нормализация ограничивает $\|W_i\|_2 = 1$ и стабилизирует обучение.

Что почитать дальше

- *Е. Е. Тыртышников.* Методы численного анализа. М.: Академия, 2007. — классический российский учебник по вычислительной линейной алгебре университетского уровня.
- *G. Strang.* Linear Algebra and Learning from Data. Wellesley-Cambridge Press, 2019. — свежая книга от автора знаменитого MIT-курса 18.06, написанная с прицелом на data science.
- *L. N. Trefethen, D. Bau.* Numerical Linear Algebra. SIAM, 1997. — классика, с очень понятным изложением SVD и итерационных методов.
- *Видеокурс «Numerical Linear Algebra», И. В. Оселедец, Сколтех* (github.com/MerkulovDaniil/nla360). Лекции, семинары, ноутбуки — из которых, в частности, и взяты eigencats для этой главы.
- *S. Brin, L. Page.* The Anatomy of a Large-Scale Hypertextual Web Search Engine. WWW Conference, 1998. — оригинальная статья про PageRank, читается легко.
- *I. Goodfellow et al.* Explaining and Harnessing Adversarial Examples. ICLR, 2015. — статья про панду-гиббона, из которой взят рис. 0.12.

Большой итоговый проект

Постройте свою рекомендательную систему для фильмов.

Возьмите датасет MovieLens-100K (открытый, 100 000 оценок, ~ 1000 пользователей $\times \sim 1700$ фильмов). Постройте матрицу «пользователь–фильм» R , где $R_{ij} \in \{1, \dots, 5\}$ — оценка, поставленная пользователем i фильму j , а 0 — «не смотрел».

1. Постройте малоранговое приближение $R \approx U\Sigma\bar{V}$ с $k = 20$ компонентами через SVD.
2. Объясните содержательно, что такое строки U (профили пользователей) и столбцы V (профили фильмов) в этом приближении. Сколько чисел теперь хранит модель вместо $1000 \cdot 1700$?
3. Для нескольких реальных пользователей выпишите топ-5 фильмов, которые рекомендация модели предлагает посмотреть. Совпадают ли они с любимыми жанрами пользователя?
4. Найдите два «противоположных» фильма с наибольшим расстоянием между их столбцами V и подумайте, можно ли увидеть содержательный смысл в этой разделяющей оси (боевик vs. мелодрама? старое vs. новое?).
5. (★) Замените SVD на алгоритм PageRank, построив граф «фильмы, которые часто оценивают одинаково». Получится ли тот же топ?

! Задачи для самостоятельной работы

1. Сложите в Python 10 000 000 копий числа 0, 1. Сколько должно получиться? Сколько получается на самом деле? Почему?
2. Вычислите $f(x) = \sqrt{x+1} - \sqrt{x}$ при $x = 10^{16}$ напрямую и через формулу $f(x) = \frac{1}{\sqrt{x+1} + \sqrt{x}}$. Сравните ответы. Какой устойчив, почему?
3. Найдите ручную SVD матрицы $A = \begin{pmatrix} 30 & 04 \\ 01 & -10 \end{pmatrix}$ и что в обоих случаях геометрически делают U, Σ, V ?
4. Возьмите свою фотографию 300×300 в градациях серого. Сделайте сжатие через SVD с рангами $k =$

- 5, 20, 50, 100. При каком k вы перестаёте узнавать самого себя?
5. Для игрушечного графа из рис. 0.11 прокрутите 5 итераций степенного метода *вручную*, начав с $r^{(0)} = (\frac{1}{6}, \dots, \frac{1}{6})$. Совпали ли ваши значения с программными после 5 шагов? а после 20?
 6. * Покажите, что для функции $\sigma(x) = \max(0, x)$ (ReLU) константа Липшица равна 1. Тот же вопрос для сигмоиды $\sigma(x) = 1/(1 + e^{-x})$: чему равна $\text{Lip}(\sigma)$ и в какой точке производная максимальна?
 7. * Найдите спектральную норму матрицы $W = \begin{pmatrix} 12 & 21 \\ 21 & 12 \end{pmatrix}$ двумя способами: (а) через определение $\|W\| = \max_{\|x\|=1} \|Wx\|$ (с использованием собственных значений $\bar{W}W$); (б) одной итерацией степенного метода (с любым стартом). Совпали ли ответы?